

Contents

Open WebUI Tor, I2P & Stealth Web Scraper — v2.1	2
☞ Open WebUI Tor, I2P & Stealth Web Scraper ☞	2
☞ Table of Contents	2
☞ Core Features	2
☞ Multi-Path Browsing	3
☞ Dynamic OpSec & JavaScript Control	4
☞ Extended Cookie Killer	4
☞ Smart Extraction	4
✂ Token & RAM Optimized	4
☞ Open WebUI Tools	4
☞ How to install	5
☞ Architecture	5
☞ Quick Start	6
☞ Complete Tutorial	7
Prerequisites	7
Folder structure	7
1. tor-proxy: Tor + Snowflake	8
2. browser-tor: anonymity via Tor	9
3. browser-clear: stealth browser with Solverrr fallback	9
4. browser-http: lightweight TLS impersonation	10
5. browser-i2p: Garlic Routing via I2P	11
6. solverrr: Cloudflare/Turnstile solver	11
7. searxng: self-hosted meta search engine	12
8. docker-compose.yml	13
9. Build and startup	14
10. Verification: test battery	15
11. Integration with OpenWebUI	16
12. Intelligent text extraction	21
13. Troubleshooting	22
14. Useful commands	23
☞ Changelog	24
v2.0 — Multi-path update	24
Appendix A — Source Code	24
browser-clear/app.py	24
browser-tor/app.py	30
browser-i2p/app.py	33
browser-http/app.py	36
openwebui-tools/web_tool.py	38
openwebui-tools/darknet_tool.py	41
searxng-config/settings.yml.example	44
docker-compose.yml	45

Open WebUI Tor, I2P & Stealth Web Scraper — v2.1

Autore: thotganesh **Data:** Ottobre 2026 **Stack:** Docker Compose · Playwright · curl_cffi · FastAPI · Tor/Snowflake · i2pd · SearXNG · Solvrr · Open WebUI

🔗 Open WebUI Tor, I2P & Stealth Web Scraper 📄

A robust, multi-path web scraping infrastructure for local AI models.

A full-stack scraping solution that feeds clean, optimized Markdown to your LLM — saving RAM, cutting token usage, bypassing anti-bot measures, and keeping your OpSec intact across **Cleagnet, Tor, I2P, and anonymous search**.

Built specifically as tools for **Open WebUI**.

📄 Table of Contents

- 📄 Core Features
 - 📄 Open WebUI Tools
 - 📄 Architecture
 - 📄 Quick Start
 - 📄 Complete Tutorial
 - [Prerequisites](#)
 - [Folder structure](#)
 - 1. tor-proxy: Tor + Snowflake
 - 2. browser-tor: anonymity via Tor
 - 3. browser-clear: stealth browser with Solvrr fallback
 - 4. browser-http: lightweight TLS impersonation
 - 5. browser-i2p: Garlic Routing via I2P
 - 6. solvrr: Cloudflare/Turnstile solver
 - 7. searxng: self-hosted meta search engine
 - 8. docker-compose.yml
 - 9. Build and startup
 - 10. Verification: test battery
 - 11. Integration with OpenWebUI
 - 12. Intelligent text extraction
 - 13. Troubleshooting
 - 14. Useful commands
-

📄 Core Features

	Feature	Description
□	Multi-Path Browsing	Five independent paths: lightweight HTTP, stealth Chromium, Tor, I2P, and anonymous search
□	Dynamic OpSec & JS Control	Turn JavaScript on/off straight from the AI prompt to reduce fingerprinting and exploit surface
□	Extended Cookie Killer	Removes Iubenda, Didomi, OneTrust, Cookiebot, Quantcast, TrustArc, Complianz, CookieYes, Termly, Klaro and more
□	Smart Extraction	Mozilla Readability + html2text deliver only the core article in pure Markdown
⚡	Token & RAM Optimized	Heavy media blocked at network level; browser-http uses curl_cffi instead of a full browser
□	Self-Hosted Search	SearXNG aggregates Google, Bing, DuckDuckGo, Brave, Wikipedia and Qwant — no API key, no cost
□	Solvrr Integration	Automatic Cloudflare/Turnstile challenge solving via a dual-browser fallback (Chromium + Camoufox)
□	Tor & I2P Search	DuckDuckGo HTML onion + Ahmia for .onion sites; Ahmia gateway for .i2p eepsites

□ Multi-Path Browsing

- **HTTP Lightweight (browser-http)** — uses curl_cffi to impersonate Chrome 124's TLS fingerprint (JA3/JA4). ~200 ms per request vs ~3-5 s for a full browser. Bypasses most “basic” anti-bot checks without the RAM cost.
- **Clear Browser (browser-clear)** — Playwright Chromium with playwright-stealth, extended cookie killer, and **automatic Solvrr fallback** for Cloudflare/Turnstile challenges. Two-pass cookie killing + scroll unlock + backdrop removal.
- **Tor Browser (browser-tor)** — hardened Firefox routed exclusively through the Tor network using the **Snowflake** pluggable transport. WebRTC disabled, privacy.resistFingerprinting, new circuit per request via stem.
- **I2P Browser (browser-i2p)** — completely isolated Garlic Routing through a local i2pd

router to explore .i2p eepsites with no clearnet exposure.

- **Search (SearXNG)** — self-hosted meta search engine, no API key, no rate limits. Replaces Google Custom Search (deprecated for new users).

▣ **Dynamic OpSec & JavaScript Control**

The Tor and I2P tools expose a `js_enabled` parameter. The AI can disable JavaScript on request (for example when the user asks for maximum security), instantly mitigating browser fingerprinting and hostile scripts on darknets.

▣ **Extended Cookie Killer**

The cookie killer now covers the major European CMPs:

- **Iubenda** (Repubblica, Corriere, GEDI)
- **Didomi** (many Italian newspapers)
- **OneTrust, Cookiebot, Quantcast, TrustArc**
- **Complianz, CookieYes, Termly, Klaro**
- **GDPR Cookie Consent, Funding Choices** (Google), **CCPA**

Some CMPs have anti-tamper protection (Iubenda on `repubblica.it` used to empty the entire page when the banner was removed with `el.remove()`). The current implementation **hides** with `display:none + visibility:hidden + opacity:0 + pointer-events:none`, then unlocks scroll, removes backdrop overlays, and hides CMP iframes. Two passes (with a 1s delay) catch banners that re-appear.

▣ **Smart Extraction**

Strips away menus, ads and sidebars, delivering only the core article as pure Markdown. Includes a fallback mechanism for homepages and search engines.

✂ **Token & RAM Optimized**

Conditionally blocks heavy media at the network level, so pages load faster and your model reads fewer tokens. The `browser-http` path avoids launching a full browser when not needed.

▣ **Open WebUI Tools**

This project ships as **two Open WebUI tools**. Ready-to-import files are in `openwebui-tools/`:

Tool	File	Purpose
Web	<code>web_tool.py</code>	Search (SearXNG) + read clearnet pages (browser-http + browser-clear)

Tool	File	Purpose
Darknet	darknet_tool.py	Search + read Tor (.onion) and I2P (.i2p)

Web exposes: - `search_web(query)` — SearXNG meta-search - `read_web(url)` — try HTTP first, fallback to full browser - `read_web_full(url)` — force full browser (Playwright + Solverr)

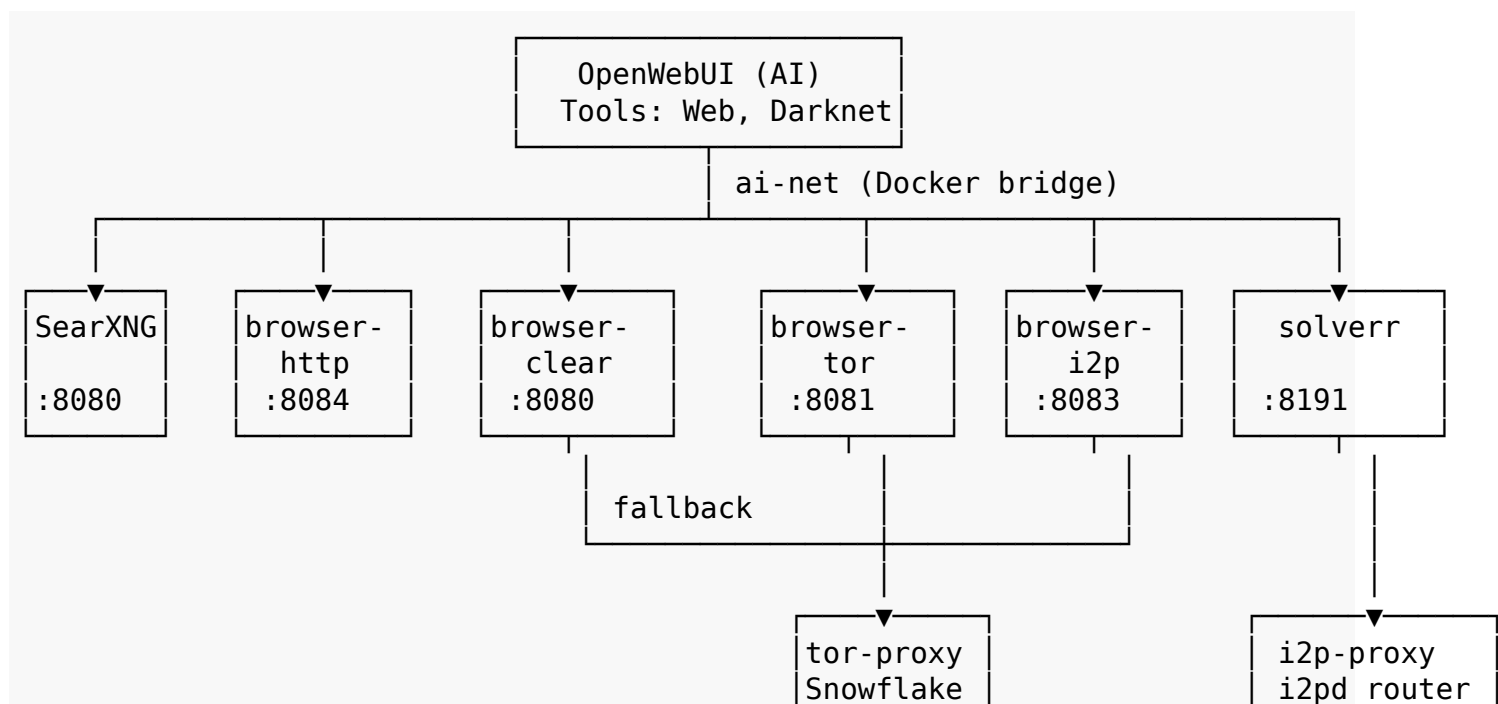
Darknet exposes: - `search_web_tor(query)` — anonymous clearnet search via Tor (DuckDuckGo HTML) - `search_onion(query)` — .onion sites via Ahmia - `search_i2p(query)` — I2P eepsites via Ahmia gateway - `read_tor(url, js_enabled)` — read a page via Tor - `read_i2p(url, js_enabled)` — read an eepsite via I2P

□ How to install

1. In OpenWebUI go to **Admin** → **Workspace** → **Tools** → **new tool**
2. Paste the content of each file (one tool per file)
3. Enable both tools in your chat and make sure your model supports function calling
4. Disable the native Web Search: **Admin** → **Settings** → **Web Search** → **OFF** (otherwise it competes with `search_web`)
5. Set the system prompt from section 11.5

[!IMPORTANT] **Both tools are required.** The Web tool talks to searxng, browser-http and browser-clear. The Darknet tool talks to browser-tor, browser-i2p and (for search) browser-http / searxng.

□ Architecture



[!NOTE] All traffic runs on the internal Docker network ai-net: no Tor, I2P, Solvrr or SearXNG port is exposed to the outside.

📦 Quick Start

Already know your way around Docker? Get the whole stack running in a few commands. Want to understand every piece? Skip to the Complete Tutorial.

Requirements: a VPS with Docker + Docker Compose, outgoing UDP open, OpenWebUI already running.

```
# 1) Clone the repository
git clone https://github.com/thotganesh/openwebui-tor-i2p-scraper.git
cd openwebui-tor-i2p-scraper

# 2) Create the shared Docker network (skip if it already exists)
docker network create ai-net

# 3) Connect OpenWebUI to the network (skip if it's already connected)
docker network connect ai-net open-webui

# 4) I2P data folder: i2pd runs as non-root and needs write access
mkdir -p i2p-data && chmod -R 777 i2p-data

# 5) Configure secrets: copy .env.example to .env and fill in your values
cp .env.example .env
# Generate the control password hash if you want a custom one:
docker run --rm debian:bookworm-slim bash -c \
  "apt-get update -qq && apt-get install -y -qq tor >/dev/null && tor --hash-password"
# Then edit .env and paste your plain-text password (TOR_CONTROL_PASSWORD)
# and the generated hash (TOR_CONTROL_HASH).

# 6) Configure SearXNG secret key
cp searxng-config/settings.yml.example searxng-config/settings.yml
# Edit searxng-config/settings.yml and replace CHANGE_ME with:
openssl rand -hex 32

# 7) Build and start everything
docker compose up -d --build

# 8) Wait for Tor: look for "Bootstrapped 100% (done)"
docker compose logs -f tor-proxy
```

[!IMPORTANT] - **Change the default password.** Never leave the placeholder values in torrc and browser-tor/app.py. - **I2P warm-up:** i2pd needs about 3-5 minutes on its first boot before .i2p sites resolve. - The first build takes a few

minutes (Snowflake compilation + Playwright image, about 2 GB).

Then: install the two tools from [openwebui-tools/](#) in OpenWebUI (see How to install) and run the test battery to verify everything.

▣ Complete Tutorial

Build the stack from scratch: SearXNG for search, a lightweight HTTP reader, a stealth browser with Solvrr, Tor with Snowflake, I2P with i2pd, all integrated as tools for a local AI on OpenWebUI.

Prerequisites

- ▣ A VPS with Docker and Docker Compose (the `docker compose` command)
- ▣ **Outgoing UDP open:** Snowflake negotiates via WebRTC/ICE, which uses UDP. If the provider blocks it, Tor stays stuck in bootstrap
- ▣ A Docker network shared with OpenWebUI, here called `ai-net`. If it doesn't exist: `docker network create ai-net`
- ▣ Free ports on the host: 8081 (browser-tor), 8082 (browser-clear), 8083 (browser-i2p), 8084 (browser-http), 8191 (solvrr), 8888 (searxng)
- ▣ OpenWebUI already running and connected to `ai-net`

Folder structure

```
~/ai-stack/
├── docker-compose.yml
├── .env.example
├── .gitignore
├── README.md
├── tor-snowflake/
│   ├── Dockerfile
│   └── torrc
├── browser-tor/
│   ├── Dockerfile
│   └── app.py
├── browser-clear/
│   ├── Dockerfile
│   └── app.py
├── browser-http/
│   ├── Dockerfile
│   ├── requirements.txt
│   └── app.py
├── browser-i2p/
│   ├── Dockerfile
│   ├── requirements.txt
│   └── app.py
├── searxng-config/
└── settings.yml
```

```
├── settings.yml.example
├── openwebui-tools/
│   ├── web_tool.py
│   └── darknet_tool.py
└── i2p-data/
```

```
mkdir -p ~/ai-stack/{tor-snowflake,browser-tor,browser-clear,browser-http,browser-i2p}
# VITAL: i2pd runs as non-root and needs permission to write its addressbook
chmod -R 777 ~/ai-stack/i2p-data
cd ~/ai-stack
```

1. tor-proxy: Tor + Snowflake

```
# --- Stage 1: build of snowflake-client ---
FROM golang:1.23-bookworm AS builder
# Allows Go to download by itself the toolchain required by the project's go.mod
ENV GOT00LCHAIN=auto
RUN git clone https://git.torproject.org/pluggable-transport/snowflake.git /tmp/snowflake
    && cd /tmp/snowflake/client \
    && go build -o /usr/local/bin/snowflake-client .
# --- Stage 2: final image, runtime only ---
FROM debian:bookworm-slim
RUN apt-get update && apt-get install -y \
    tor obfs4proxy ca-certificates curl \
    && rm -rf /var/lib/apt/lists/*
COPY --from=builder /usr/local/bin/snowflake-client /usr/local/bin/snowflake-client
RUN chmod +x /usr/local/bin/snowflake-client
EXPOSE 9150
CMD ["tor", "-f", "/etc/tor/torrc"]
```

1.1 Dockerfile (multi-stage) — tor-snowflake/Dockerfile

[!NOTE] The Snowflake repo requires a very recent Go (1.25.x). With `golang:1.23` + `GOT00LCHAIN=auto`, Go downloads the right toolchain by itself. The final stage contains only the compiled binary: the image stays lightweight.

1.2 ControlPort password The ControlPort is used to ask Tor for a new circuit (NEWNYM) before each request. Generate the hash:

```
docker run --rm debian:bookworm-slim bash -c \
    "apt-get update -qq && apt-get install -y -qq tor >/dev/null && tor --hash-password"
# the last printed line is the hash, like: 16:AF97... (copy it into .env as TOR_CONTROL_HASH)
```

Copy the hash (16:...) into `.env` as `TOR_CONTROL_HASH`. The same password in plain text goes into `.env` as `TOR_CONTROL_PASSWORD`.

```
SocksPort 0.0.0.0:9150
UseBridges 1
ClientTransportPlugin snowflake exec /usr/local/bin/snowflake-client \
  -url https://snowflake-broker.torproject.net.global.prod.fastly.net/ \
  -front cdn.sstatic.net \
  -ice stun:stun.l.google.com:19302,stun:stun.antisip.com:3478,stun:stun.bluesip.net:
Bridge snowflake 192.0.2.3:80 2B280B23E1107BB62ABFC40DDCC8824814F80A72 fingerprint=2B
ControlPort 0.0.0.0:9151
# HashedControlPassword is injected automatically via docker-compose and .env
```

1.3 torrc — tor-snowflake/torrc

[!WARNING] The `-ice` and `Bridge` lines are **one single line each**. The IP 192.0.2.3 in the `Bridge` line is a placeholder from the official documentation: with Snowflake it doesn't matter, because the client connects to the broker via domain fronting and then negotiates with a volunteer proxy via WebRTC.

2. browser-tor: anonymity via Tor

□ `browser-tor/`

```
FROM mcr.microsoft.com/playwright/python:v1.48.0-jammy
WORKDIR /app
RUN pip install fastapi uvicorn httpx playwright==1.48.0 stem readability-lxml html2text
COPY app.py /app/app.py
EXPOSE 8081
CMD ["uvicorn", "app:app", "--host", "0.0.0.0", "--port", "8081"]
```

What it does (`browser-tor/app.py`):

- □ **Hardened Firefox**: `privacy.resistFingerprinting` + letterboxing reduce the browser's uniqueness; viewport 1000x900, UTC timezone and en-US language are values common among Tor users
 - □ **WebRTC disabled**: with WebRTC enabled the real IP leaks via ICE/STUN, bypassing the SOCKS proxy
 - □ **New circuit per request**: NEWNYM via ControlPort (Playwright doesn't support authentication on SOCKS5, so random username/password aren't used; stem requires a literal IP, hence `socket.gethostbyname`)
 - □ **Text extraction**: Readability + html2text fallback (see section 12)
-

3. browser-clear: stealth browser with Solverr fallback

□ `browser-clear/`

```
FROM mcr.microsoft.com/playwright/python:v1.48.0-jammy
WORKDIR /app
RUN pip install fastapi uvicorn playwright-stealth httpx playwright==1.48.0 readability-lxml
COPY app.py /app/app.py
```

EXPOSE 8080

CMD ["uvicorn", "app:app", "--host", "0.0.0.0", "--port", "8080"]

What it does ([browser-clear/app.py](#)):

- **Stealth:** `Stealth().use_async(async_playwright())` (playwright-stealth 2.x) masks the typical automation signals
- **Extended cookie killer:** hides iubenda, Didomi, OneTrust, Cookiebot, Quantcast, TrustArc, Complianz, CookieYes, Termly, Klaro, GDPR Cookie Consent, Funding Choices, CCPA banners. Two passes with a 1s delay. Unlocks scroll, removes backdrops and CMP iframes.
- **Solvrr fallback:** if a Cloudflare/Turnstile challenge is detected (based on page title and body content), the request is re-issued through Solvrr on `http://solvrr:8191/v1`.
- **Banner detection heuristic:** if the extracted text looks like a cookie banner (short + contains multiple CMP-specific strings), the request is retried via Solvrr.
- **WebRTC intentionally enabled:** a real Chrome always has it on; here the real IP isn't a secret to protect
- **block_media enabled by default** for speed and RAM saving; with `false` it loads everything like a real user (useful against aggressive anti-bots)

Real-world performance (verified with `curl`):

Site	CMP	Before	After
repubblica.it	iubenda	797 chars	103,691 chars
corriere.it	Didomi	—	165,908 chars
ilpost.it	Simple	—	32,727 chars
it.wikipedia.org/wiki/Italia	None	—	250,610 chars

4. browser-http: lightweight TLS impersonation

▢ [browser-http/](#)

FROM python:3.11-slim

WORKDIR /app

RUN apt-get update && apt-get install -y --no-install-recommends gcc \
&& rm -rf /var/lib/apt/lists/*

COPY requirements.txt .

RUN pip install --no-cache-dir -r requirements.txt

COPY app.py .

EXPOSE 8084

CMD ["uvicorn", "app:app", "--host", "0.0.0.0", "--port", "8084"]

What it does ([browser-http/app.py](#)):

- **Uses `curl_cffi`** with `impersonate="chrome124"` to send a request with a real Chrome TLS fingerprint (JA3/JA4), bypassing “basic” anti-bot checks without launching a browser
- **Same extraction pipeline** as `browser-clear`: Readability + `html2text` fallback

- **Cheapest path:** ~200 ms and ~50 MB RAM per request vs ~3-5 s and ~300-500 MB for a browser context
-

5. browser-i2p: Garlic Routing via I2P

□ `browser-i2p/`

```
FROM mcr.microsoft.com/playwright/python:v1.40.0-jammy
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
RUN playwright install firefox
COPY . .
EXPOSE 8083
CMD ["uvicorn", "app:app", "--host", "0.0.0.0", "--port", "8083"]
```

What it does (`browser-i2p/app.py`):

- **Garlic Routing via i2pd:** all traffic goes through the dedicated i2p-proxy container (the i2pd router), never through your real IP
 - **JavaScript control:** `js_enabled=false` turns JavaScript off completely, recommended on I2P against fingerprinting and hostile scripts
 - **“Airbag” error handling:** a `try...except` block gracefully handles offline .i2p eepsites (very common on I2P) instead of crashing
 - **Text extraction:** same Readability + `html2text` pipeline as the other browsers
- [!NOTE] I2P is intrinsically slow: that’s why the tool uses a longer timeout (150 s) and encourages `block_media=true`.
-

6. solverr: Cloudflare/Turnstile solver

Solverr is an improved fork of FlareSolverr that uses **two browsers** (Chromium + Camoufox) and switches between them automatically. Maintains persistent sessions, cleans them up, and answers the FlareSolverr-compatible API on port 8191.

```
solverr:
  image: ghcr.io/unseensnick/solverr:latest
  container_name: solverr
  networks: [ai-net]
  ports:
    - "127.0.0.1:8191:8191"
  shm_size: 512mb
  restart: unless-stopped
```

Why it’s needed: playwright-stealth doesn’t solve Cloudflare Turnstile or “Just a moment” pages. Solverr handles them and returns the HTML + cookies to browser-clear.

How browser-clear calls it:

```

async def solve_with_solvr(url: str) -> str:
    async with httpx.AsyncClient(timeout=90) as client:
        r = await client.post(
            "http://solvr:8191/v1",
            json={"cmd": "request.get", "url": url, "maxTimeout": 60000},
        )
        data = r.json()
        if data.get("status") == "ok":
            return data.get("solution", {}).get("response", "")
    return ""

```

[!NOTE] Solvr's IP reputation matters more than anything else. A VPS IP will fail more challenges than a residential one. If you need residential proxies in the future, Solvr supports PROXY_URL.

7. searxng: self-hosted meta search engine

SearXNG aggregates results from Google, Bing, DuckDuckGo, Brave, Wikipedia, Qwant and many more, with **no API key, no rate limits, and full privacy**.

```

searxng:
  image: searxng/searxng:latest
  container_name: searxng
  networks: [ai-net]
  ports:
    - "127.0.0.1:8888:8080"
  volumes:
    - ./searxng-config:/etc/searxng:rw
  environment:
    - SEARXNG_BASE_URL=http://searxng:8080/
  restart: unless-stopped

```

Configuration — [searxng-config/settings.yml](#):

- **formats:** [html, json] — **required**, otherwise OpenWebUI can't query it
- **limiter:** false — disables the internal rate limiter (otherwise it blocks OpenWebUI after a few queries)
- **method:** GET — compatible with the OpenWebUI client
- **default_lang:** it-IT — Italian results by default
- **Engines enabled:** google, bing, duckduckgo, brave, wikipedia, qwant

Generate a secret key:

```

cp searxng-config/settings.yml.example searxng-config/settings.yml
openssl rand -hex 32
# Paste the result into secret_key in settings.yml

```

[!IMPORTANT] **Why not Google Custom Search?** Google has removed the "Search the entire web" option for new users and the Custom Search JSON API is being deprecated for new clients. SearXNG is the sustainable alternative: free, unlimited, self-hosted, and already integrated with OpenWebUI.

8. docker-compose.yml

□ docker-compose.yml

```
services:
  tor-proxy:
    build: ./tor-snowflake
    container_name: tor-proxy
    networks: [ai-net]
    environment:
      - TOR_CONTROL_HASH=${TOR_CONTROL_HASH:?Set TOR_CONTROL_HASH in .env}
    command: ["sh", "-c", "exec tor -f /etc/tor/torrc --HashedControlPassword \"$$TOR"]
    volumes:
      - ./tor-snowflake/torrc:/etc/tor/torrc:ro
    restart: unless-stopped

  browser-tor:
    build: ./browser-tor
    container_name: browser-tor
    networks: [ai-net]
    environment:
      - TOR_CONTROL_PASSWORD=${TOR_CONTROL_PASSWORD:?Set TOR_CONTROL_PASSWORD in .env}
    depends_on: [tor-proxy]
    ports:
      - "127.0.0.1:8081:8081"
    restart: unless-stopped

  browser-clear:
    build: ./browser-clear
    container_name: browser-clear
    networks: [ai-net]
    ports:
      - "127.0.0.1:8082:8080"
    restart: unless-stopped

  browser-http:
    build: ./browser-http
    container_name: browser-http
    networks: [ai-net]
    ports:
      - "127.0.0.1:8084:8084"
    restart: unless-stopped

  i2p-proxy:
    image: purplei2p/i2pd:latest
    container_name: i2p-proxy
    networks: [ai-net]
    command: --httpproxy.address=0.0.0.0 --socksproxy.address=0.0.0.0
```

```

volumes:
  - ./i2p-data:/home/i2pd/data
restart: unless-stopped

browser-i2p:
  build: ./browser-i2p
  container_name: browser-i2p
  networks: [ai-net]
  depends_on: [i2p-proxy]
  ports:
    - "127.0.0.1:8083:8083"
  restart: unless-stopped

solverr:
  image: ghcr.io/unseensnick/solverr:latest
  container_name: solverr
  networks: [ai-net]
  ports:
    - "127.0.0.1:8191:8191"
  shm_size: 512mb
  restart: unless-stopped

searxng:
  image: searxng/searxng:latest
  container_name: searxng
  networks: [ai-net]
  ports:
    - "127.0.0.1:8888:8080"
  volumes:
    - ./searxng-config:/etc/searxng:rw
  environment:
    - SEARXNG_BASE_URL=http://searxng:8080/
  restart: unless-stopped

networks:
  ai-net:
    external: true

```

[!IMPORTANT] ai-net is external: true: the network already exists and is shared with OpenWebUI. The ports are bound to 127.0.0.1: **never publish Tor's 9150/9151, I2P's 4444/4447, Solverr's 8191 or SearXNG's 8888.**

9. Build and startup

```

cd ~/ai-stack
docker compose build
docker compose up -d

```

```
docker compose ps                # all eight containers "Up", no restart loop
docker compose logs -f tor-proxy # wait for "Bootstrapped 100% (done)"
```

[!NOTE] The first build takes a few minutes (Snowflake compilation + Playwright image download, about 2 GB).

[!IMPORTANT] **I2P warm-up time:** the i2pd router needs about 3-5 minutes on its very first boot to build its tunnels and populate the addressbook. Test it with `http://stats.i2p` (see the test battery) only after that.

10. Verification: test battery

```
# 1) Tor's SOCKS works and the IP is anonymous
docker run --rm --network ai-net curlimages/curl -s --socks5-hostname tor-proxy:9150

# 2) browser-clear: must show IsTor:false with the VPS's real IP
curl -s -X POST http://127.0.0.1:8082/browse -H "Content-Type: application/json" \
  -d '{"url": "https://check.torproject.org/api/ip"}'

# 3) browser-tor: must show IsTor:true with an exit node IP
curl -s -X POST http://127.0.0.1:8081/browse -H "Content-Type: application/json" \
  -d '{"url": "https://check.torproject.org/api/ip"}'

# 4) browser-tor on a real .onion (DuckDuckGo)
curl -s -X POST http://127.0.0.1:8081/browse -H "Content-Type: application/json" \
  -d '{"url": "http://duckduckgogg42xjoc72x3sjasowoarfbgcmvfimaftt6twagswzczad.onion"}'

# 5) Circuit rotation: NEWNYM has a rate limit of ~10 s
curl -s -X POST http://127.0.0.1:8081/browse -H "Content-Type: application/json" \
  -d '{"url": "https://check.torproject.org/api/ip"}'; echo; sleep 12
curl -s -X POST http://127.0.0.1:8081/browse -H "Content-Type: application/json" \
  -d '{"url": "https://check.torproject.org/api/ip"}'

# 6) WebRTC diagnostic test (WebRTC deliberately re-enabled: must show the real IP leaked)
curl -s http://127.0.0.1:8081/webrtc-leak-check

# 7) browser-i2p on an eepsite, JavaScript OFF (wait ~5 min after first boot)
curl -s -X POST http://127.0.0.1:8083/browse -H "Content-Type: application/json" \
  -d '{"url": "http://stats.i2p", "js_enabled": false}'

# 8) browser-http: fast HTTP path with TLS impersonation
curl -s -X POST http://127.0.0.1:8084/browse -H "Content-Type: application/json" \
  -d '{"url": "https://example.com"}'

# 9) Solvrr health check
curl -s http://127.0.0.1:8191/

# 10) SearXNG health check + JSON search
```

```
curl -s "http://127.0.0.1:8888/search?q=test&format=json" | head -c 300
```

11) Cookie killer on a CMP-heavy site

```
curl -s -X POST http://127.0.0.1:8082/browse -H "Content-Type: application/json" \
-d '{"url": "https://www.repubblica.it"}' | python3 -c "import sys,json;print('Len:"
```

Test	Expected result
Direct SOCKS	{"IsTor":true,"IP":"..."}
browser-clear	IsTor:false with the VPS's real IP
browser-tor	IsTor:true, exit node IP different from the VPS's
browser-tor on .onion	HTTP 200 with title and text of the page
Circuit rotation	Different exit IPs between the two requests
webrtc-leak-check	leaked_ips with the real IP: proof that WebRTC must stay disabled
browser-i2p on stats.i2p	HTTP 200 with title and text of the eepsite (after warm-up)
browser-http	HTTP 200 with title and text, in ~200 ms
Solvert	{"msg":"FlareSolvert is ready!"...}
SearXNG	JSON with "results": [...]
Cookie killer on Repubblica	Len > 50,000 (before the fix: ~800)

[!NOTE] Always use real URLs in tests: a placeholder URL returns a 404 that looks like a code problem.

11. Integration with OpenWebUI

```
docker inspect open-webui --format '{{json .NetworkSettings.Networks}}' # ai-net mu
docker network connect ai-net open-webui # only if m
docker exec open-webui curl -s -m 10 http://browser-clear:8080/health # {"status"
docker exec open-webui curl -s -m 10 http://browser-tor:8081/health # {"status"
docker exec open-webui curl -s -m 10 http://browser-i2p:8083/health # {"status"
docker exec open-webui curl -s -m 10 http://browser-http:8084/health # {"status"
docker exec open-webui curl -s -m 10 http://searxng:8080/ # HTML
docker exec open-webui curl -s -m 10 http://solvert:8191/ # {"msg":"P
```

11.1 Network and reachability

11.2 Tool 1: Web (search + reader) In OpenWebUI: **Admin** → **Workspace** → **Tools** → **new tool**. Name: Web. Paste [openwebui-tools/web_tool.py](#).

```
"""
title: Web (Search + Reader)
description: Cerca su internet (SearXNG) e legge qualsiasi pagina web normale. Scegli
author: NeoPC
version: 1.0.0
```

```

required_open_webui_version: 0.3.0
"""

import httpx

class Tools:
    def __init__(self):
        self.searxng_url = "http://searxng:8080"
        self.http_url = "http://browser-http:8084"
        self.clear_url = "http://browser-clear:8080"

    async def search_web(self, query: str, num: int = 5) -> str:
        """
        Cerca su internet usando SearXNG (aggregatore di Google, Bing, DuckDuckGo,
        Brave, Wikipedia, Qwant). Restituisce titolo, estratto e URL dei risultati.

        Usa questo quando l'utente chiede di cercare, trovare o sapere qualcosa
        su internet. Dopo aver ottenuto gli URL, usa read_web per leggere le
        pagine interessanti.

        :param query: Termine di ricerca (es. "ultime notizie intelligenza artificiale")
        :param num: Numero massimo di risultati (default 5, max 10).
        """
        num = min(max(num, 1), 10)
        try:
            async with httpx.AsyncClient(timeout=30) as client:
                r = await client.get(
                    f"{self.searxng_url}/search",
                    params={"q": query, "format": "json", "language": "it-IT", "safesearch": "0"}
                )
                data = r.json()
                results = data.get("results", [])[:num]
                if not results:
                    return f"Nessun risultato trovato per: {query}"
                out = [f"**Risultati per: {query}\n"]
                for i, item in enumerate(results, 1):
                    out.append(
                        f"{i}. **{item.get('title', 'Senza titolo')}**\n"
                        f"    {item.get('content', '')[:300]}\n"
                        f"    URL: {item.get('url', '')}\n"
                        f"    (fonte: {item.get('engine', '')})"
                    )
                return "\n\n".join(out)
        except Exception as e:
            return f"Errore durante la ricerca: {str(e)}"

    async def read_web(self, url: str) -> str:
        """

```

Legge una pagina web normale (http/https). Prova prima HTTP veloce (curl_cffi con impersonificazione TLS), poi browser completo se il contenuto e' troppo corto o c'e' un errore.

:param url: URL completo da visitare (es. https://example.com).
"""

```
result = await self._post(self.http_url, {"url": url})
if not result or result.get("error"):
    return await self.read_web_full(url)
if len(result.get("text", "")) < 200:
    return await self.read_web_full(url)
return self._format(result)
```

```
async def read_web_full(self, url: str) -> str:
    """
```

Legge una pagina web usando un browser completo (Playwright + Solverr).

:param url: URL completo da visitare.
"""

```
result = await self._post(self.clear_url, {"url": url})
return self._format(result)
```

```
async def _post(self, base_url: str, payload: dict) -> dict:
    try:
```

```
        async with httpx.AsyncClient(timeout=150) as client:
            r = await client.post(f"{base_url}/browse", json=payload)
            return r.json()
```

```
    except Exception as e:
        return {"error": str(e), "text": "", "title": ""}
```

```
def _format(self, data: dict) -> str:
```

```
    if not data:
        return "Errore: nessuna risposta dal servizio."
```

```
    if data.get("error"):
        return f"Errore: {data['error']}"
```

```
    text = data.get("text", "")
```

```
    if not text:
```

```
        return f"Titolo: {data.get('title', '')}\nURL: {data.get('final_url', '')}"
```

```
    return f"Titolo: {data.get('title', '')}\nURL finale: {data.get('final_url', '')}"
```

11.3 Tool 2: Darknet (Tor + I2P) Name: Darknet. Paste openwebui-tools/darknet_tool.py.

```
"""
```

title: Darknet (Tor + I2P)

description: Cerca e legge su Tor (.onion) e I2P (.i2p). Include ricerca web anonima

author: NeoPC

version: 2.0.0

required_open_webui_version: 0.3.0

```
"""
```

```

import httpx
from urllib.parse import quote

class Tools:
    def __init__(self):
        self.tor_url = "http://browser-tor:8081"
        self.i2p_url = "http://browser-i2p:8083"
        self.http_url = "http://browser-http:8084"

    async def search_web_tor(self, query: str) -> str:
        """
        Cerca sul web normale (clearnet) in modo anonimo tramite Tor,
        usando DuckDuckGo HTML (non-JavaScript).

        :param query: Termine di ricerca.
        """
        encoded = quote(query)
        search_url = (
            "https://duckduckgogg42xjoc72x3sjasowoarfbgcmvfimaftt6twagswzczad.onion/h"
            f"?q={encoded}"
        )
        result = await self._post(self.tor_url, {"url": search_url, "js_enabled": False})
        return "[TOR SEARCH | DuckDuckGo HTML] " + self._format(result)

    async def search_onion(self, query: str) -> str:
        """
        Cerca specificamente su siti .onion (servizi nascosti Tor) usando Ahmia.

        :param query: Termine di ricerca.
        """
        encoded = quote(query)
        search_url = (
            "http://juhanurmihxlp77nkq76byazcldy2hlmovfu2epvl5ankdibsot4csyd.onion"
            f"/search/?q={encoded}"
        )
        result = await self._post(self.tor_url, {"url": search_url, "js_enabled": False})
        return "[TOR ONION SEARCH | Ahmia] " + self._format(result)

    async def search_i2p(self, query: str) -> str:
        """
        Cerca su eepsite I2P usando Ahmia (gateway clearnet).

        :param query: Termine di ricerca.
        """
        encoded = quote(query)
        search_url = f"https://ahmia.fi/i2p/search/?q={encoded}"
        result = await self._post(self.http_url, {"url": search_url})
        return "[I2P SEARCH | Ahmia Gateway] " + self._format(result)

```

```

async def read_tor(self, url: str, js_enabled: bool = False) -> str:
    """
    Legge una pagina tramite rete Tor.

    :param url: URL completo (.onion o clearnet).
    :param js_enabled: True attiva JavaScript. Default False.
    """
    result = await self._post(self.tor_url, {"url": url, "js_enabled": js_enabled})
    prefix = "[TOR] " if js_enabled else "[TOR | JS OFF] "
    return prefix + self._format(result)

async def read_i2p(self, url: str, js_enabled: bool = False) -> str:
    """
    Legge un eepsite I2P (.i2p).

    :param url: URL completo .i2p.
    :param js_enabled: True attiva JavaScript. Default False.
    """
    result = await self._post(self.i2p_url, {"url": url, "js_enabled": js_enabled})
    prefix = "[I2P] " if js_enabled else "[I2P | JS OFF] "
    return prefix + self._format(result)

async def _post(self, base_url: str, payload: dict) -> dict:
    try:
        async with httpx.AsyncClient(timeout=200) as client:
            r = await client.post(f"{base_url}/browse", json=payload)
            return r.json()
    except Exception as e:
        return {"error": str(e), "text": "", "title": ""}

def _format(self, data: dict) -> str:
    if not data:
        return "Errore: nessuna risposta dal servizio."
    if data.get("error"):
        return f"Errore: {data['error']}"
    text = data.get("text", "")
    if not text:
        return f"Titolo: {data.get('title', '')}\nURL: {data.get('final_url', '')}"
    return f"Titolo: {data.get('title', '')}\nURL finale: {data.get('final_url', '')}"

```

11.4 Avoiding conflicts with native Web Search Disable the integrated Web Search (**Admin Panel** → **Settings** → **Web Search** → **OFF**): with competing tools, small models choose inconsistently.

11.5 System Prompt In **Admin** → **Settings** → **General** → **System Prompt**:

Sei un assistente con accesso a due tool per navigare il web: "Web" e "Darknet".

TOOL "Web" – per il web normale (clearnet):

- `search_web(query)`: cerca su internet (SearXNG aggrega Google/Bing/DDG/Brave). USA QUESTO per qualsiasi domanda che richiede informazioni attuali, notizie, fatti verificabili, o quando l'utente dice "cerca", "trova", "cercami". Dopo aver ottenuto gli URL, usa `read_web` per leggere le fonti rilevanti.
- `read_web(url)`: legge una pagina http/https normale. Prova prima HTTP veloce, poi browser completo automaticamente. USA QUESTO per la maggior parte dei siti.
- `read_web_full(url)`: forza il browser completo con Solvrr. USA QUESTO se `read_web` fallisce o sospetti Cloudflare/JavaScript pesante.

TOOL "Darknet" – per Tor e I2P:

- `search_web_tor(query)`: cerca sul web normale in modo anonimo via Tor.
- `search_onion(query)`: cerca specificamente su siti .onion (Ahmia).
- `search_i2p(query)`: cerca eepsite I2P (Ahmia gateway).
- `read_tor(url, js_enabled)`: legge siti .onion o naviga in anonimato.
- `read_i2p(url, js_enabled)`: legge eepsite .i2p.

REGOLE GENERALI:

1. Non inventare URL. Se non conosci il sito, usa PRIMA `search_web` (o `search_onion` per .onion).
2. Non provare a scrapare google.com: usa `search_web`.
3. Per domande fattuali semplici, rispondi direttamente senza cercare.
4. Per domande su eventi recenti o dati aggiornati, cerca SEMPRE.
5. Quando citi informazioni da una pagina, indica sempre la fonte (titolo + URL).
6. Per URL .onion usa `read_tor`. Per URL .i2p usa `read_i2p`. Non mescolare.
7. Per cercare su Tor, usa `search_onion` per .onion o `search_web_tor` per clearnet.
8. I2P ha pochi motori di ricerca: `search_i2p` via Ahmia e' il piu' affidabile.

If the model doesn't call the tools, check that they are enabled in the chat and that the model supports function calling (model settings → Capabilities).

12. Intelligent text extraction

Mechanisms present across the browsers:

1. **Resource blocking** (`block_media`): `page.route("**/*.{png,jpg,...}")` stops images, fonts and videos before they download → faster pages and less memory. On sites with serious anti-bot, disable it: a Chrome that never loads images is anomalous behavior.
2. **Extended cookie killer**: hides the containers of the main CMPs (Iubenda, Didomi, OneTrust, Cookiebot, Quantcast, TrustArc, Complianz, CookieYes, Termly, Klaro, GDPR Cookie Consent, Funding Choices, CCPA) with `display:none + visibility:hidden + opacity:0`, then unlocks scroll and removes backdrops and CMP iframes. Two passes with a 1s delay.
3. **Two-level extraction**: level 1 Readability (structured article); level 2 fallback html2text on all the HTML (homepages, indexes, search results). You never get a hard error.
4. **domcontentloaded + fixed wait (2500 ms)** instead of `networkidle`: ad-filled sites

never reach the “silent network” and would go into timeout.

5. **Solvert retry on banner detection:** if the extracted text looks like a cookie banner (< 3000 chars AND contains at least 2 CMP-specific strings), the request is re-issued via Solvert.

[!WARNING] Some CMPs have anti-tamper protection: on repubblica.it (lubenda), removing the banner with `el.remove()` emptied the entire page (HTML from 496,335 to 15 characters). The current implementation **hides** with `display:none` instead of removing, which is why the text extraction now returns 100,000+ characters on that site.

13. Troubleshooting

Error / symptom	Cause	Solution
I2P container Restarting (139)	Permission denied on /home/i2pd/data	<code>chmod -R 777 i2p-data</code>
I2P NS_ERROR_UNKNOWN_PROXY_HOST	The i2p-proxy container is down or unreachable	Check permissions on i2p-data, then <code>docker compose restart i2p-proxy</code>
I2P Host not found in addressbook	The router is still warming up	Wait ~5 minutes, then test <code>http://stats.i2p</code>
I2P tool returns an error / timeout	Eepsite offline (very common) or slow tunnels	Retry later or try another eepsite
Error parsing Bridge address 'auto'	Incorrect or leftover torrc syntax	Rewrite the torrc from scratch
general SOCKS server failure (obfs4)	obfs4 bridge with made-up fingerprint/cert	Use Snowflake with the official bridge line
invalid go version '1.25.12' / requires go >= 1.25.12	Image's Go too old	Multi-stage with golang and <code>ENV GOT00LCHAIN=auto</code>
pip3: not found	Node.js variant Playwright image	Use mcr.microsoft.com/playwright/python
No module named 'playwright'	The Python image doesn't include the package	Explicit <code>pip install playwright==1.48.0</code>
Executable doesn't exist ...	Python package newer than the image's browsers	Pin <code>playwright==</code> to the image version
chrome-headless-shell port is already allocated	Host port already used	Change the mapping (e.g. 8082:8080)
cannot import name 'stealth_async'	playwright-stealth 2.x has a new API	<code>Stealth().use_async(async_playwright)</code>
RTCPeerConnection is not defined	With WebRTC disabled the API doesn't exist	Leak test with WebRTC re-enabled (dedicated endpoint)
Browser does not support socks5 proxy authentication	Playwright doesn't support user/password on SOCKS5	Isolate circuits with <code>ControlPort</code> and <code>NEWNYM</code>

Error / symptom	Cause	Solution
Invalid IP address: tor-proxy	stem requires a literal IP	<code>socket.gethostbyname("tor-proxy")</code>
Document is empty / Unparseable	Readability doesn't find an article	<code>html2text</code> fallback on all the HTML
Page emptied after the cookie killer	The CMP detects the banner removal (anti-tamper)	Hide with <code>display:none</code> instead of <code>remove()</code>
Cannot read properties of null (reading 'style')	<code>document.body</code> absent during a reload	<code>if (document.body) + try/except</code>
Cookie banner still in the text	Case-sensitive selectors or CMP not in the list	Add the site's CMP IDs (or selectors with the <code>i</code> flag)
Error: GOOGLE_API_KEY not configured	You were using an old Google Search tool	Not needed anymore — SearXNG replaces it
SearXNG returns HTML instead of JSON	formats: not set in <code>settings.yml</code>	Ensure <code>formats: [html, json]</code> and <code>limiter: false</code>
SearXNG blocks after a few queries	The internal limiter is active	Set <code>limiter: false</code> in <code>settings.yml</code>
OpenWebUI can't reach searxng	OpenWebUI not on ai-net	<code>docker network connect ai-net open-webui</code>
Solvrr Challenge not solved	VPS IP is blocked by Cloudflare	Use residential proxies (future enhancement)
404 response in tests	Placeholder URL taken literally	Use a real URL
curl responds with 0 bytes	Error 500 hidden by <code>curl -s</code>	<code>curl -v</code> and <code>docker compose logs <service></code>

14. Useful commands

```

cd ~/ai-stack
docker compose ps                                # container status
docker compose logs -f browser-tor                # Tor browser logs in real time
docker compose logs -f i2p-proxy                  # monitor I2P tunnels
docker compose logs -f browser-clear              # cookie killer + Solvrr fallback
docker compose logs -f solvrr                     # challenge solving
docker compose logs -f searxng                     # search engine
docker compose logs browser-clear --tail 30 | grep -i warn

# after modifying an app.py / Dockerfile:
docker compose build browser-clear browser-tor browser-i2p browser-http
docker compose up -d browser-clear browser-tor browser-i2p browser-http

docker compose restart tor-proxy                  # restart only Tor
docker compose restart i2p-proxy                  # restart only I2P
docker compose restart searxng                    # restart only SearXNG
docker compose restart solvrr                     # restart only Solvrr

```

docker compose down

stop everything (the ai-ne

📄 Changelog

v2.0 — Multi-path update

- Added **browser-http** (curl_cffi with TLS impersonation) — ~10x faster than a full browser for simple pages
- Added **solvrr** (Cloudflare/Turnstile solver, FlareSolvrr-compatible API)
- Added **searxng** (self-hosted meta search engine, replaces Google Custom Search)
- Extended **cookie killer** with Iubenda, Didomi, Complianz, CookieYes, Termly, Klaro, GDPR Cookie Consent, Funding Choices, CCPA
- Added **double-pass cookie killer**, scroll unlock, backdrop removal, CMP iframe hiding
- Added **Solvrr fallback** in browser-clear for Cloudflare/Turnstile challenges
- Added **banner detection heuristic** with Solvrr retry
- Unified OpenWebUI tools from 3 to 2: **Web** (search + reader) and **Darknet** (Tor + I2P)
- Added **Tor search** via DuckDuckGo HTML onion and Ahmia onion search
- Added **I2P search** via Ahmia gateway
- Fixed Unicode encoding in README (emojis and box drawing characters)

📄 If this project helps you, consider leaving a star on the repo!

Appendix A — Source Code

browser-clear/app.py

```
from fastapi import FastAPI
from pydantic import BaseModel
from playwright.async_api import async_playwright
from playwright_stealth import Stealth
from readability import Document
import html2text
import httpx
import re

app = FastAPI()

SOLVERR_URL = "http://solvrr:8191/v1"

class BrowseRequest(BaseModel):
    url: str
    wait_selector: str | None = None
    extract_text: bool = True
```

```
block_media: bool = True
```

```
# =====  
# COOKIE KILLER ESTESO  
# =====  
COOKIE_KILLER_JS = ""  
async () => {  
  const CMP_SELECTORS = [  
    // OneTrust  
    '#onetrust-banner-sdk', '#onetrust-consent-sdk',  
    // Cookiebot  
    '#cookiebot', '#CybotCookiebotDialog', '#CybotCookiebotDialogBodyUnderlay',  
    // Quantcast  
    '.qc-cmp2-container', '#qc-cmp2-container',  
    // TrustArc  
    '#truste-consent-track', '.truste_box_overlay', '.truste_overlay',  
    // Generic  
    '.cc-window', '.fc-consent-root', '#cookie-notice',  
    '.paywall', '[class*="cookie-banner"]', '[id*="cookie-consent"]',  
    // Iubenda (Repubblica, Corriere, GEDI)  
    '#iubenda-cs-banner', '.iubenda-cs-container', '.iubenda-cs-visible',  
    '[class*="iubenda"]', 'iframe[src*="iubenda"]',  
    // Didomi (molti giornali italiani)  
    '#didomi-host', '#didomi-notice', '.didomi-popup-backdrop',  
    '.didomi-notice-popup', '[class*="didomi"]',  
    // Complianz  
    '.cmplz-cookiebanner', '.cmplz-manage-consent',  
    // CookieYes  
    '.cky-consent-container', '.cky-modal',  
    // GDPR Cookie Consent  
    '.cli-modal-backdrop', '#cookie-law-info-bar',  
    // Termly  
    '.termly-cookie-policy', '#termly-code-snippet-support',  
    // Klaro  
    '.klaro .cookie-modal', '.klaro .cookie-notice',  
    // CCPA  
    '#ccpa-banner', '#ccpa-notice',  
    // Funding Choices (Google)  
    '.fc-consent-root', '.fc-dialog-overlay',  
    // Other  
    '#gdpr-banner', '.gdpr-banner', '#cookie-law-info-bar',  
  ];  
  
  // 1. Nascondi (display:none) invece di rimuovere, per evitare anti-tamper  
  CMP_SELECTORS.forEach(sel => {  
    document.querySelectorAll(sel).forEach(el => {  
      el.style.setProperty('display', 'none', 'important');  
      el.style.setProperty('visibility', 'hidden', 'important');  
    });  
  });  
}
```

```

        el.style.setProperty('opacity', '0', 'important');
        el.style.setProperty('pointer-events', 'none', 'important');
    });
});

// 2. Sblocca lo scroll (molti CMP lo bloccano)
document.documentElement.style.setProperty('overflow', 'auto', 'important');
document.body.style.setProperty('overflow', 'auto', 'important');
document.body.style.setProperty('position', 'static', 'important');
document.body.style.setProperty('height', 'auto', 'important');

// 3. Rimuovi backdrop/overlay generici (position:fixed + z-index alto)
document.querySelectorAll('body > div, body > aside, body > section').forEach(el => {
    try {
        const s = window.getComputedStyle(el);
        const z = parseInt(s.zIndex) || 0;
        if (s.position === 'fixed' && z > 1000 && el.offsetHeight > window.innerHeight) {
            el.style.setProperty('display', 'none', 'important');
        }
    } catch (e) { /* ignore */ }
});

// 4. Rimuovi eventuali iframe CMP
document.querySelectorAll('iframe').forEach(iframe => {
    const src = (iframe.src || '').toLowerCase();
    if (src.includes('iubenda') || src.includes('didomi') || src.includes('onetrust') ||
        src.includes('cookiebot') || src.includes('consent')) {
        iframe.style.setProperty('display', 'none', 'important');
    }
});

return 'cookie-killer: done';
}
"""

# =====
# RILEVAMENTO CHALLENGE
# =====
def is_challenge_page(html: str, title: str) -> bool:
    indicators = [
        "just a moment",
        "checking your browser",
        "verify you are human",
        "cf-chl",
        "turnstile",
        "attention required",
        "enable javascript and cookies to continue",
    ]

```

```

combined = (html + (title or "")).lower()
return any(ind in combined for ind in indicators)

# =====
# RILEVAMENTO BANNER CMP RESIDUO
# =====
def is_cookie_banner(text: str) -> bool:
    """Euristicica: se il testo sembra un banner cookie invece di contenuto reale."""
    if len(text) > 3000:
        return False
    lower = text.lower()
    indicators = [
        "consenso all'utilizzo di cookie",
        "cookie policy",
        "finalità diverse da quelle strettamente necessarie",
        "pannello delle preferenze pubblicitarie",
        "rifiuta e abbonati",
        "accetta e chiudi",
        "gestione cookie",
        "we use cookies",
        "accept all cookies",
    ]
    hits = sum(1 for ind in indicators if ind in lower)
    return hits >= 2

# =====
# SOLVERR
# =====
async def solve_with_solvr(url: str) -> str:
    async with httpx.AsyncClient(timeout=90) as client:
        try:
            r = await client.post(
                SOLVERR_URL,
                json={"cmd": "request.get", "url": url, "maxTimeout": 60000},
            )
            data = r.json()
            if data.get("status") == "ok":
                return data.get("solution", {}).get("response", "")
        except Exception:
            pass
    return ""

# =====
# ESTRAZIONE TESTO
# =====
def pulisci_testo(html_content: str) -> str:

```

```

h = html2text.HTML2Text()
h.ignore_links = False
h.ignore_images = True
h.body_width = 0

try:
    doc = Document(html_content)
    testo_pulito = h.handle(doc.summary()).strip()
    if testo_pulito and len(testo_pulito) > 200:
        return testo_pulito
except Exception:
    pass

try:
    return h.handle(html_content).strip()
except Exception as e:
    return f"Errore nell'estrazione del testo: {e}"

# =====
# ENDPOINT PRINCIPALE
# =====
@app.post("/browse")
async def browse(req: BrowseRequest):
    async with Stealth().use_async(async_playwright()) as p:
        browser = await p.chromium.launch(
            headless=True,
            args=[
                "--disable-blink-features=AutomationControlled",
                "--no-sandbox",
                "--disable-dev-shm-usage",
            ],
        )
        context = await browser.new_context(
            user_agent="Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36",
            viewport={"width": 1366, "height": 768},
            locale="it-IT",
        )
        page = await context.new_page()

        if req.block_media:
            await page.route(
                "**/*.{png,jpg,jpeg,gif,svg,woff,woff2,mp4,webm}",
                lambda route: route.abort(),
            )

        html = ""
        title = ""
        try:

```

```

await page.goto(req.url, wait_until="domcontentloaded", timeout=60000)
await page.wait_for_timeout(2500)

# Cookie killer (1a passata)
await page.evaluate(COOKIE_KILLER_JS)

# Seconda passata dopo un breve delay (alcuni CMP riappaiono)
await page.wait_for_timeout(1000)
await page.evaluate(COOKIE_KILLER_JS)

html = await page.content()
title = await page.title()
except Exception as e:
    await browser.close()
    return {
        "title": "",
        "final_url": req.url,
        "text": "",
        "error": f"Errore Playwright: {e}",
    }

await browser.close()

# Fallback Solverr se challenge rilevata
if is_challenge_page(html, title):
    solverr_html = await solve_with_sloverr(req.url)
    if solverr_html:
        html = solverr_html
        try:
            doc = Document(html)
            title = doc.title() or title
        except Exception:
            pass
    else:
        return {
            "title": title,
            "final_url": req.url,
            "text": "",
            "error": "Challenge rilevata ma Solverr non ha risolto",
        }

result = {"title": title, "final_url": req.url}
if req.extract_text:
    text = pulisci_testo(html)

# Se il testo sembra un banner CMP residuo, ritenta con Solverr
if is_cookie_banner(text):
    solverr_html = await solve_with_sloverr(req.url)
    if solverr_html:

```

```

        text2 = pulisci_testo(solverr_html)
        if len(text2) > len(text):
            text = text2
        try:
            doc = Document(solverr_html)
            title = doc.title() or title
        except Exception:
            pass

    result["text"] = text

    return result

@app.get("/health")
async def health():
    return {"status": "ok"}

```

browser-tor/app.py

```

import os
import asyncio
import socket
from fastapi import FastAPI
from pydantic import BaseModel
from playwright.async_api import async_playwright
from stem import Signal
from stem.control import Controller
from readability import Document
import html2text

app = FastAPI()

class BrowseRequest(BaseModel):
    url: str
    wait_selector: str | None = None
    extract_text: bool = True
    block_media: bool = True
    js_enabled: bool = True

TOR_PROXY = {"server": "socks5://tor-proxy:9150"}
CONTROL_PASSWORD = os.environ.get("TOR_CONTROL_PASSWORD")

FIREFOX_PREFS = {
    "media.peerconnection.enabled": False,

```

```

    "network.dns.disablePrefetch": True,
    "network.prefetch-next": False,
    "geo.enabled": False,
    "privacy.resistFingerprinting": True,
    "privacy.resistFingerprinting.letterboxing": True,
    "privacy.spoof_english": 2,
    "webgl.disabled": True,
    "dom.battery.enabled": False,
    "device.sensors.enabled": False,
    "media.navigator.enabled": False,
}

# Extended cookie killer (aligned with browser-clear)
COOKIE_KILLER_JS = ""
async () => {
    const CMP_SELECTORS = [
        '#onetrust-banner-sdk', '#onetrust-consent-sdk',
        '#cookiebot', '#CybotCookiebotDialog',
        '.qc-cmp2-container', '#qc-cmp2-container',
        '#truste-consent-track', '.truste_box_overlay',
        '.cc-window', '.fc-consent-root', '#cookie-notice',
        '.paywall', '[class*="cookie-banner"]', '[id*="cookie-consent"]',
        '#iubenda-cs-banner', '.iubenda-cs-container', '.iubenda-cs-visible',
        '[class*="iubenda"]', 'iframe[src*="iubenda"]',
        '#didomi-host', '#didomi-notice', '.didomi-popup-backdrop',
        '[class*="didomi"]',
        '.cmplz-cookiebanner', '.cmplz-manage-consent',
        '.cky-consent-container', '.cky-modal',
        '.cli-modal-backdrop', '#cookie-law-info-bar',
        '.termly-cookie-policy',
        '.klaro .cookie-modal', '.klaro .cookie-notice',
        '#ccpa-banner',
        '.fc-dialog-overlay',
        '#gdpr-banner', '.gdpr-banner',
    ];
    CMP_SELECTORS.forEach(sel => {
        document.querySelectorAll(sel).forEach(el => {
            el.style.setProperty('display', 'none', 'important');
            el.style.setProperty('visibility', 'hidden', 'important');
        });
    });
    document.documentElement.style.setProperty('overflow', 'auto', 'important');
    if (document.body) {
        document.body.style.setProperty('overflow', 'auto', 'important');
    }
    return 'cookie-killer: done';
}
""

```

```

def request_new_circuit():
    try:
        tor_proxy_ip = socket.gethostbyname("tor-proxy")
        with Controller.from_port(address=tor_proxy_ip, port=9151) as controller:
            controller.authenticate(password=CONTROL_PASSWORD)
            controller.signal(Signal.NEWNYM)
            print(f"[info] New circuit requested via {tor_proxy_ip}")
    except Exception as e:
        print(f"[warn] Could not request new circuit: {e}")

def pulisci_testo(html_content: str) -> str:
    h = html2text.HTML2Text()
    h.ignore_links = False
    h.ignore_images = True
    h.body_width = 0

    try:
        doc = Document(html_content)
        testo_pulito = h.handle(doc.summary()).strip()
        if testo_pulito:
            return testo_pulito
    except Exception:
        pass

    try:
        return h.handle(html_content).strip()
    except Exception as e:
        return f"Extraction error: {e}"

@app.post("/browse")
async def browse(req: BrowseRequest):
    request_new_circuit()
    await asyncio.sleep(1)

    async with async_playwright() as p:
        browser = await p.firefox.launch(
            headless=True,
            proxy=TOR_PROXY,
            firefox_user_prefs=FIREFOX_PREFS,
        )
        context = await browser.new_context(
            viewport={"width": 1000, "height": 900},
            timezone_id="UTC",
            locale="en-US",
            java_script_enabled=req.js_enabled,
        )

```

```

page = await context.new_page()

if req.block_media:
    await page.route(
        "**/*.{png,jpg,jpeg,gif,svg,woff,woff2,mp4,webm}",
        lambda route: route.abort(),
    )

# Navigation with error handling (fix: .onion unreachable no longer 500)
try:
    await page.goto(req.url, wait_until="domcontentloaded", timeout=60000)
    await page.wait_for_timeout(2500)
except Exception as e:
    await browser.close()
    return {
        "error": f"Navigation failed: {e}",
        "title": "",
        "final_url": req.url,
        "text": "",
    }

if req.js_enabled:
    try:
        await page.evaluate(COOKIE_KILLER_JS)
    except Exception as e:
        print(f"[warn] Cookie killer failed: {e}")

result = {"title": await page.title(), "final_url": page.url}
if req.extract_text:
    result["text"] = pulisci_testo(await page.content())

await browser.close()
return result

@app.get("/health")
async def health():
    return {"status": "ok"}

```

browser-i2p/app.py

```

import asyncio
from fastapi import FastAPI
from pydantic import BaseModel
from playwright.async_api import async_playwright
from readability import Document
import html2text

```

```

app = FastAPI()

class BrowseRequest(BaseModel):
    url: str
    extract_text: bool = True
    block_media: bool = True
    js_enabled: bool = True

I2P_PROXY = {"server": "http://i2p-proxy:4444"}

FIREFOX_PREFS = {
    "media.peerconnection.enabled": False,
    "network.dns.disablePrefetch": True,
    "network.prefetch-next": False,
    "geo.enabled": False,
    "privacy.resistFingerprinting": True,
    "webgl.disabled": True,
    "dom.battery.enabled": False,
    "device.sensors.enabled": False,
}

# Extended cookie killer (aligned with browser-clear)
COOKIE_KILLER_JS = """
async () => {
    const CMP_SELECTORS = [
        '#onetrust-banner-sdk', '#onetrust-consent-sdk',
        '#cookiebot', '#CybotCookiebotDialog',
        '.qc-cmp2-container', '#qc-cmp2-container',
        '#truste-consent-track', '.truste_box_overlay',
        '.cc-window', '.fc-consent-root', '#cookie-notice',
        '.paywall', '[class*="cookie-banner"]', '[id*="cookie-consent"]',
        '#iubenda-cs-banner', '.iubenda-cs-container', '.iubenda-cs-visible',
        '[class*="iubenda"]', 'iframe[src*="iubenda"]',
        '#didomi-host', '#didomi-notice', '.didomi-popup-backdrop',
        '[class*="didomi"]',
        '.cmplz-cookiebanner', '.cmplz-manage-consent',
        '.cky-consent-container', '.cky-modal',
        '.cli-modal-backdrop', '#cookie-law-info-bar',
        '.termly-cookie-policy',
        '.klaro .cookie-modal', '.klaro .cookie-notice',
        '#ccpa-banner',
        '.fc-dialog-overlay',
        '#gdpr-banner', '.gdpr-banner',
    ];
    CMP_SELECTORS.forEach(sel => {
        document.querySelectorAll(sel).forEach(el => {

```

```

        el.style.setProperty('display', 'none', 'important');
        el.style.setProperty('visibility', 'hidden', 'important');
    });
});
document.documentElement.style.setProperty('overflow', 'auto', 'important');
if (document.body) {
    document.body.style.setProperty('overflow', 'auto', 'important');
}
return 'cookie-killer: done';
}
"""

```

```

def pulisci_testo(html_content: str) -> str:
    h = html2text.HTML2Text()
    h.ignore_links = False
    h.ignore_images = True
    h.body_width = 0

    try:
        doc = Document(html_content)
        testo = h.handle(doc.summary()).strip()
        if testo:
            return testo
    except Exception:
        pass

    try:
        return h.handle(html_content).strip()
    except Exception as e:
        return f"Extraction error: {e}"

```

```

@app.post("/browse")
async def browse(req: BrowseRequest):
    async with async_playwright() as p:
        browser = await p.firefox.launch(
            headless=True,
            proxy=I2P_PROXY,
            firefox_user_prefs=FIREFOX_PREFS,
        )
        context = await browser.new_context(
            viewport={"width": 1000, "height": 900},
            timezone_id="UTC",
            locale="en-US",
            java_script_enabled=req.js_enabled,
        )
        page = await context.new_page()

```

```

if req.block_media:
    await page.route(
        "**/*.{png,jpg,jpeg,gif,svg,woff,woff2,mp4,webm}",
        lambda route: route.abort(),
    )

# AIRBAG: catch navigation errors
try:
    await page.goto(req.url, wait_until="domcontentloaded", timeout=120000)
    await page.wait_for_timeout(2500)
except Exception as e:
    await browser.close()
    return {
        "title": "I2P Network Error",
        "final_url": req.url,
        "text": f"I2P site currently unreachable or router not yet synced. De
    }

if req.js_enabled:
    try:
        await page.evaluate(COOKIE_KILLER_JS)
    except Exception as e:
        print(f"[warn] Cookie killer failed: {e}")

result = {"title": await page.title(), "final_url": page.url}
if req.extract_text:
    result["text"] = pulisci_testo(await page.content())

await browser.close()
return result

@app.get("/health")
async def health():
    return {"status": "ok"}

```

browser-http/app.py

```

from fastapi import FastAPI
from pydantic import BaseModel
from curl_cffi import requests
from readability import Document
import html2text

app = FastAPI()

class Request(BaseModel):

```

```

url: str

def extract_text(html: str) -> str:
    """Two-level extraction: Readability first, html2text fallback."""
    h = html2text.HTML2Text()
    h.ignore_links = False
    h.ignore_images = True
    h.body_width = 0

    try:
        doc = Document(html)
        clean = h.handle(doc.summary()).strip()
        if clean and len(clean) > 200:
            return clean
    except Exception:
        pass

    try:
        return h.handle(html).strip()
    except Exception as e:
        return f"Extraction error: {e}"

@app.post("/browse")
async def browse(req: Request):
    try:
        response = requests.get(
            req.url,
            impersonate="chrome124",
            timeout=30,
            allow_redirects=True,
        )

        if response.status_code != 200:
            return {
                "error": f"HTTP {response.status_code}",
                "text": "",
                "title": "",
                "final_url": req.url,
            }

        # response.url reflects redirects (curl_cffi behavior)
        final_url = str(response.url) if response.url else req.url
        html = response.text

        try:
            doc = Document(html)
            title = doc.title() or ""

```

```

except Exception:
    title = ""

text = extract_text(html)

return {
    "title": title,
    "text": text,
    "final_url": final_url, # no 10k truncation
                        # URL after redirects
}

except Exception as e:
    return {
        "error": str(e),
        "text": "",
        "title": "",
        "final_url": req.url,
    }

@app.get("/health")
async def health():
    return {"status": "ok"}

```

openwebui-tools/web_tool.py

```

"""
title: Web (Search + Reader)
description: Cerca su internet (SearXNG) e legge qualsiasi pagina web normale. Scegli
author: NeoPC
version: 1.0.0
required_open_webui_version: 0.3.0
"""

import httpx

class Tools:
    def __init__(self):
        self.searxng_url = "http://searxng:8080"
        self.http_url = "http://browser-http:8084"
        self.clear_url = "http://browser-clear:8080"

        # =====
        # RICERCA WEB
        # =====

    async def search_web(self, query: str, num: int = 5) -> str:

```

```

"""
Cerca su internet usando SearXNG (aggregatore di Google, Bing, DuckDuckGo,
Brave, Wikipedia, Qwant). Restituisce titolo, estratto e URL dei risultati.

Usa questo quando l'utente chiede di cercare, trovare o sapere qualcosa
su internet. Dopo aver ottenuto gli URL, usa read_web per leggere le
pagine interessanti.

:param query: Termine di ricerca (es. "ultime notizie intelligenza artificiale")
:param num: Numero massimo di risultati (default 5, max 10).
"""

```

```

num = min(max(num, 1), 10)
try:
    async with httpx.AsyncClient(timeout=30) as client:
        r = await client.get(
            f"{self.searxng_url}/search",
            params={
                "q": query,
                "format": "json",
                "language": "it-IT",
                "safesearch": 0,
            },
        )
        data = r.json()
        results = data.get("results", [])[:num]

        if not results:
            return f"Nessun risultato trovato per: {query}"

        out = [f"**Risultati per:** {query}\n"]
        for i, item in enumerate(results, 1):
            title = item.get("title", "Senza titolo")
            content = item.get("content", "")[:300]
            url = item.get("url", "")
            engine = item.get("engine", "")
            out.append(
                f"{i}. **{title}**\n"
                f"    {content}\n"
                f"    URL: {url}\n"
                f"    (fonte: {engine})"
            )

        return "\n\n".join(out)

except Exception as e:
    return f"Errore durante la ricerca: {str(e)}"

# =====
# LETTURA PAGINE WEB

```

```

# =====

async def read_web(self, url: str) -> str:
    """
    Legge una pagina web normale (http/https). Prova prima HTTP veloce
    (curl_cffi con impersonificazione TLS), poi browser completo se il
    contenuto e' troppo corto o c'e' un errore.

    USA QUESTA per la maggior parte dei siti.

    :param url: URL completo da visitare (es. https://example.com).
    """
    # Tentativo 1: HTTP veloce
    result = await self._post(self.http_url, {"url": url})

    if not result or result.get("error"):
        return await self.read_web_full(url)

    text = result.get("text", "")
    if len(text) < 200:
        # Sospetto JS-only o challenge
        return await self.read_web_full(url)

    return self._format(result)

async def read_web_full(self, url: str) -> str:
    """
    Legge una pagina web usando un browser completo (Playwright con stealth
    e fallback automatico a Solvrr per Cloudflare/Turnstile).

    USA QUESTA se read_web fallisce, se la pagina richiede JavaScript, o
    se sospetti Cloudflare/Turnstile.

    :param url: URL completo da visitare (es. https://example.com).
    """
    result = await self._post(self.clear_url, {"url": url})
    return self._format(result)

# =====
# HELPER INTERNI
# =====

async def _post(self, base_url: str, payload: dict) -> dict:
    try:
        async with httpx.AsyncClient(timeout=150) as client:
            r = await client.post(f"{base_url}/browse", json=payload)
            return r.json()
    except Exception as e:
        return {"error": str(e), "text": "", "title": ""}

```

```

def _format(self, data: dict) -> str:
    if not data:
        return "Errore: nessuna risposta dal servizio."

    if data.get("error"):
        return f"Errore: {data['error']}"

    title = data.get("title", "Senza titolo")
    url = data.get("final_url", "")
    text = data.get("text", "")

    if not text:
        return f"Titolo: {title}\nURL: {url}\n\nNessun contenuto estratto."

    return f"Titolo: {title}\nURL finale: {url}\n\n{text[:6000]}"

```

openwebui-tools/darknet_tool.py

```

"""
title: Darknet (Tor + I2P)
description: Cerca e legge su Tor (.onion) e I2P (.i2p). Include ricerca web anonima
author: NeoPC
version: 2.0.0
required_open_webui_version: 0.3.0
"""

import httpx
from urllib.parse import quote

class Tools:
    def __init__(self):
        self.tor_url = "http://browser-tor:8081"
        self.i2p_url = "http://browser-i2p:8083"
        self.http_url = "http://browser-http:8084"

    # =====
    # RICERCA SU TOR
    # =====

    async def search_web_tor(self, query: str) -> str:
        """
        Cerca sul web normale (clearnet) in modo anonimo tramite Tor,
        usando DuckDuckGo HTML (non-JavaScript). Restituisce risultati
        di ricerca per il web accessibile via Tor.

        :param query: Termine di ricerca.
        """

```

```

        encoded = quote(query)
        search_url = (
            "https://duckduckgogg42xjoc72x3sjasowoarfbgcmvfimaftt6twagswzczad.onion/h
            f"?q={encoded}"
        )
        result = await self._post(
            self.tor_url,
            {"url": search_url, "js_enabled": False, "block_media": True},
        )
        return "[TOR SEARCH | DuckDuckGo HTML] " + self._format(result)

    async def search_onion(self, query: str) -> str:
        """
        Cerca specificamente su siti .onion (servizi nascosti Tor) usando Ahmia.
        Usa questo per trovare hidden services .onion.

        :param query: Termine di ricerca.
        """
        encoded = quote(query)
        search_url = (
            "http://juhanurmihxlp77nkq76byazcldy2hlmovfu2epvl5ankdibsot4csyd.onion"
            f"/search/?q={encoded}"
        )
        result = await self._post(
            self.tor_url,
            {"url": search_url, "js_enabled": False, "block_media": True},
        )
        return "[TOR ONION SEARCH | Ahmia] " + self._format(result)

# =====
# RICERCA SU I2P
# =====

    async def search_i2p(self, query: str) -> str:
        """
        Cerca su eepsite I2P usando Ahmia (che indicizza anche .i2p).
        Nota: questa ricerca avviene via clearnet (Ahmia e' un gateway),
        non via rete I2P. Restituisce link a eepsite .i2p.

        :param query: Termine di ricerca.
        """
        encoded = quote(query)
        search_url = f"https://ahmia.fi/i2p/search/?q={encoded}"
        result = await self._post_http({"url": search_url})
        return "[I2P SEARCH | Ahmia Gateway] " + self._format(result)

# =====
# LETTURA PAGINE
# =====

```

```

async def read_tor(self, url: str, js_enabled: bool = False) -> str:
    """
    Legge una pagina tramite rete Tor. Usa questo per siti .onion o quando
    l'utente chiede esplicitamente la massima anonimità'.

    :param url: URL completo (.onion o clearnet).
    :param js_enabled: True attiva JavaScript (compatibilità'). False per
                      massima sicurezza e anti-fingerprinting. Default False.
    """
    result = await self._post(
        self.tor_url,
        {"url": url, "js_enabled": js_enabled, "block_media": True},
    )
    prefix = "[TOR] " if js_enabled else "[TOR | JS OFF] "
    return prefix + self._format(result)

async def read_i2p(self, url: str, js_enabled: bool = False) -> str:
    """
    Legge un eepsite I2P (.i2p). JavaScript disabilitato di default per
    sicurezza (I2P è intrinsecamente lento e ostile agli script).

    :param url: URL completo .i2p.
    :param js_enabled: True per attivare JavaScript (sconsigliato).
                      Default False.
    """
    result = await self._post(
        self.i2p_url,
        {"url": url, "js_enabled": js_enabled, "block_media": True},
    )
    prefix = "[I2P] " if js_enabled else "[I2P | JS OFF] "
    return prefix + self._format(result)

# =====
# HELPER INTERNI
# =====

async def _post(self, base_url: str, payload: dict) -> dict:
    try:
        async with httpx.AsyncClient(timeout=200) as client:
            r = await client.post(f"{base_url}/browse", json=payload)
            return r.json()
    except Exception as e:
        return {"error": str(e), "text": "", "title": ""}

async def _post_http(self, payload: dict) -> dict:
    try:
        async with httpx.AsyncClient(timeout=60) as client:
            r = await client.post(f"{self.http_url}/browse", json=payload)
            return r.json()

```

```

except Exception as e:
    return {"error": str(e), "text": "", "title": ""}

def _format(self, data: dict) -> str:
    if not data:
        return "Errore: nessuna risposta dal servizio."

    if data.get("error"):
        return f"Errore: {data['error']}"

    title = data.get("title", "Senza titolo")
    url = data.get("final_url", "")
    text = data.get("text", "")

    if not text:
        return f"Titolo: {title}\nURL: {url}\n\nNessun contenuto estratto."

    return f"Titolo: {title}\nURL finale: {url}\n\n{text[:6000]}"

```

searxng-config/settings.yml.example

```

use_default_settings: true

general:
  debug: false
  instance_name: "SearXNG for OpenWebUI"
  enable_metrics: false

search:
  safe_search: 0
  autocomplete: ""
  default_lang: "it-IT"
  formats:
    - html
    - json

server:
  secret_key: "CHANGE_ME_generate_with_openssl_rand_hex_32"
  limiter: false
  image_proxy: false
  method: "GET"

ui:
  static_use_hash: true

redis:
  url: false

```

```
outgoing:
  request_timeout: 10.0
  max_request_timeout: 15.0
  pool_connections: 100
  pool_maxsize: 20
  enable_http2: true
```

```
engines:
- name: google
  engine: google
  shortcut: go
  disabled: false
- name: bing
  engine: bing
  shortcut: bi
  disabled: false
- name: duckduckgo
  engine: duckduckgo
  shortcut: ddg
  disabled: false
- name: brave
  engine: brave
  shortcut: br
  disabled: false
- name: wikipedia
  engine: wikipedia
  shortcut: wp
  disabled: false
- name: qwant
  engine: qwant
  shortcut: qw
  disabled: false
```

docker-compose.yml

```
services:
  tor-proxy:
    build: ./tor-snowflake
    container_name: tor-proxy
    networks: [ai-net]
    environment:
      - TOR_CONTROL_HASH=${TOR_CONTROL_HASH:?Set TOR_CONTROL_HASH in .env}
    command: ["sh", "-c", "exec tor -f /etc/tor/torrc --HashedControlPassword \"$$TOR"]
    volumes:
      - ./tor-snowflake/torrc:/etc/tor/torrc:ro
    restart: unless-stopped

  browser-tor:
```

```

build: ./browser-tor
container_name: browser-tor
networks: [ai-net]
environment:
  - TOR_CONTROL_PASSWORD=${TOR_CONTROL_PASSWORD:?Set TOR_CONTROL_PASSWORD in .env}
depends_on: [tor-proxy]
ports:
  - "127.0.0.1:8081:8081"
restart: unless-stopped

browser-clear:
build: ./browser-clear
container_name: browser-clear
networks: [ai-net]
ports:
  - "127.0.0.1:8082:8080"
restart: unless-stopped

i2p-proxy:
image: purplei2p/i2pd:latest
container_name: i2p-proxy
networks: [ai-net]
command: --httpproxy.address=0.0.0.0 --socksproxy.address=0.0.0.0
restart: unless-stopped
volumes:
  - ./i2p-data:/home/i2pd/data

browser-i2p:
build: ./browser-i2p
container_name: browser-i2p
networks: [ai-net]
depends_on: [i2p-proxy]
ports:
  - "127.0.0.1:8083:8083"
restart: unless-stopped

browser-http:
build: ./browser-http
container_name: browser-http
networks: [ai-net]
ports:
  - "127.0.0.1:8084:8084"
restart: unless-stopped

solverrr:
image: ghcr.io/unseensnick/solverrr:latest
container_name: solverrr
networks: [ai-net]
ports:

```

```
- "127.0.0.1:8191:8191"
shm_size: 512mb
restart: unless-stopped

searxng:
  image: searxng/searxng:latest
  container_name: searxng
  networks: [ai-net]
  ports:
    - "127.0.0.1:8888:8080"
  volumes:
    - ./searxng-config:/etc/searxng:rw
  environment:
    - SEARXNG_BASE_URL=http://searxng:8080/
  restart: unless-stopped

networks:
  ai-net:
    external: true
```